

Pay with Quai

Accept crypto payments on Quai in three steps. Funds go straight to your wallet — the contract holds nothing — and you get a signed webhook when a payment confirms.

AUDIENCE	NETWORK	VERSION
Integrating engineers	Quai (single zone)	v1

1. You register an order on-chain



2. Customer pays it



3. We POST you a signed webhook

Before you start

Your platform operator gives you three things at onboarding:

You receive	What it's for
<code>PAYWITHQUAI_ADDRESS</code>	The contract address you register orders on.
<code>webhookSecret</code>	Secret key to verify webhooks are really from us — shown once, store it safely .
<code>merchantId</code>	Your platform id, e.g. <code>mch_ab12...</code> .

Two rules that matter:

- **Same zone.** Quai is sharded. Your payout wallet, your customers, and the contract must all be in the **same zone** (the contract address prefix tells you which, e.g. `0x00...` = Cyprus-1). Value can't move between zones.
- **Amounts are in the token's smallest unit**, as strings/bigint — never floats. For a 6-decimal stablecoin, `$25.00` = `25000000`. Quai is `token = address(0)`.

Step 1 — Register the order (from your backend)

Register every expected payment before the customer pays. Broadcast **from your payout wallet** — that wallet becomes both your merchant identity and where the money lands.

```
import { Contract, Wallet, JsonRpcProvider, id as keccak } from 'quais';

const provider = new JsonRpcProvider('https://orchard.rpc.quai.network', undefined, {
  usePathing: true });
const wallet = new Wallet(BACKEND_PRIVATE_KEY, provider); // your payout wallet
const pay = new Contract(PAYWITHQUAI_ADDRESS, PAYWITHQUAI_ABI, wallet);

const orderId = keccak(`ord_${yourInternalRef}_${Date.now()}`); // unique bytes32
const amount = 25000000n; // $25.00, smallest unit

// registerOrder(orderId, token, amount, expiry) expiry 0 = never expires
const tx = await pay.registerOrder(orderId, TOKEN_ADDRESS, amount, 0n);
await tx.wait();
```

- `token`: an allowlisted ERC-20 address, or `address(0)` for native QUAI.
- The platform fee is locked into the order automatically here — you don't pass it.

Step 2 — Customer pays

Point the customer at any checkout you own. Payment is always a single contract call.

ERC-20 — approve, then pay

```
await token.approve(PAYWITHQUAI_ADDRESS, amount); // customer approves exact amount
await pay.payOrder(MERCHANT_ADDRESS, orderId); // customer pays
```

Native QUAI — send exact value

```
await pay.payOrderNative(MERCHANT_ADDRESS, orderId, { value: amount });
```

The contract splits off the fee, forwards the rest to your wallet, and marks the order settled — all in one transaction. A second payment reverts, so double-fulfillment is impossible.

Step 3 — Receive & verify the webhook

When the payment is final, we POST one webhook to your endpoint:

```
{
  "id": "0xabc...:3",
  "type": "payment.confirmed",
  "data": {
    "merchantId": "mch_ab12...",
    "orderId": "0x...",
    "payer": "0x...",
    "token": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "amount": "2500000", // gross the payer sent
    "fee": "250000", // platform fee
    "net": "24750000", // what you actually received
    "txHash": "0x...",
    "blockNumber": 1234567
  }
}
```

Always verify the signature before trusting a webhook

A verified `payment.confirmed` is the only signal you should fulfil an order on — then credit `net`.

```
import express from 'express';
import { createHmac, timingSafeEqual } from 'node:crypto';

const app = express();
// IMPORTANT: hash the raw body — not parsed-then-restringified JSON.
app.post('/webhooks/paywithquai', express.raw({ type: 'application/json' })), (req, res) => {
  const header = req.header('X-PayWithQuai-Signature') ?? ''; // "t=...,v1=..."
  const raw = req.body.toString('utf8');
  const { t, v1 } = Object.fromEntries(header.split(',').map((p) => p.split('=')));

  const now = Math.floor(Date.now() / 1000);
  if (Math.abs(now - Number(t)) > 300) return res.sendStatus(400); // reject stale (>5 min)

  const expected = createHmac('sha256', WEBHOOK_SECRET).update(`${t}.${raw}`).digest();
  const received = Buffer.from(v1, 'hex');
  const ok = received.length === expected.length && timingSafeEqual(received, expected);
  if (!ok) return res.sendStatus(401);

  const { data } = JSON.parse(raw);
  // Idempotent: if you've already fulfilled this orderId/txHash, just ack and return.
  fulfillOrder(data.orderId, data.net);

  res.sendStatus(200); // 2xx within 10s = success. Anything else is retried.
};
```

Your endpoint must be **https**, publicly reachable, respond **2xx within 10s**, and be **idempotent** — the same payment can arrive more than once.

Quick reference

Check an order's status yourself (fallback if you miss a webhook)

```
GET /v1/orders/0x<merchant>/0x<orderId>
→ { merchant, orderId, token, amount, feeBps, expiry, settled, webhook }
```

Or read the chain directly — `getOrder(merchant, orderId)` / `isSettled(merchant, orderId)`. `settled == true` is the final word on payment.

Common reverts to surface to customers

Revert	Meaning
<code>OrderAlreadySettled</code>	Already paid.
<code>OrderExpired</code>	Past the expiry timestamp.
<code>IncorrectNativeValue</code>	<code>msg.value</code> $\neq$ the order amount (native path).
<code>WrongPaymentPath</code>	Used the ERC-20 call on a native order, or vice-versa.
<code>OrderNotFound</code>	No such order for that merchant.

Managing orders (all from your payout wallet)

- `cancelOrder(orderId)` — cancel an unpaid order and free the id.
- `purgeSettledOrder(orderId)` — reclaim storage 1 day after settlement.
- Use a fresh `orderId` per payment; reusing ids makes status lookups ambiguous.

Testing on testnet

```
npx hardhat run scripts/deploy.js --network cyprus1 # deploys contract + mock stablecoin
npx hardhat run scripts/payDemo.js --network cyprus1 # mints, registers, approves, pays
```

Run a relay locally against the same RPC. For local endpoints only, set

`WEBHOOK_ALLOW_INSECURE_URLS=true` to allow `http://localhost` webhook targets.

Pay with Quai · Merchant Integration Guide · v1 · Amounts are smallest-unit integers on-chain and decimal strings in webhook payloads. The verified `payment.confirmed` webhook is the source of truth for fulfilment.